IMESHDATA SCRIPT INTERFACE

The `IMeshData` interface represents a series of long-lived vertex buffers for use with the renderer. It includes position, texture co-ordinate, color and index data. The data lives persistently on the GPU, ensuring it is a much more efficient way to draw complex meshes than the renderer `drawMesh()` method, which uploads all the provided data to the GPU with every call; in contrast the `drawMeshData()` method accepts an `IMeshData` and re-uses its content without having to upload any more data.

This interface is created using the renderer `createMeshData()` method. Once created the number of vertices or indices cannot be changed.

For code examples using IMeshData, see the Draw mesh data (JavaScript) or Draw mesh data (TypeScript) examples.

> A newly created IMeshData starts in an empty state. Once some data has been written to its data arrays, there must be at least one call to mark the vertex data changed and another call to mark the index data changed, otherwise it will not upload any data to the GPU and the corresponding GPU buffers will remain empty.

IMeshData APIs

`vertexCount`
`indexCount`
Read-only numbers with the vertex and index count the mesh data was created with.

`positions`

A Float32Array of vertex positions, with three components per vertex in X, Y, Z order (hence the length of this array is `3 * vertexCount`).

`texCoords`
A Float32Array of texture co-ordinates, with two components per vertex in X, Y order (hence the length of this array is `2 * vertexCount`).

`colors`
A Float16Array (where supported) or Float32Array of vertex colors, with four components per vertex in R, G, B, A order (hence the length of this array is `4 * vertexCount`).

> Note the use of Float16Array depends on hardware support, so the type may vary between devices even when the browser supports Float16Array.

`indices`
A Uint16Array or Uint32Array of indices. Uint16Array is used when the vertex count fits within a 16-bit index, otherwise Uint32Array is used. Construct always uses indexed rendering, so this must be provided; if indices are not actually used, fill the buffer with incrementing numbers to render the provided vertices in order. As the index buffer specifies triangles, its length should be a multiple of three. Note that indices refer to the index of a vertex, rather than a direct index in to a data array like `positions`. For example a positions array with elements x1, y1, z1, x2, y2, z2 has six elements but defines two vertices, and so index 1 refers to the second vertex.

`debugLabel`
A read-only string with the `debugLabel` option specified when creating the mesh data.

`markDataChanged(bufferType, start, end)`
Mark a range of data changed in a specific data buffer. `bufferType` must be one of `"positions"`, `"texCoords"`, `"colors"` or `"indices"`. `start` and `end` are optional: if omitted, the entire buffer is marked changed; if specified then only the given range (in vertices or indices) is marked changed up to but excluding `end`. `end` may also be omitted to update from

`start` to the end of the buffer. Once data is marked as changed, it will be uploaded to the GPU the next time this mesh data is drawn.

> When changing the mesh data, try to only mark the affected buffers changed for the minimal affected ranges, as it will reduce the amount of data that is sent to the GPU which improves performance.

`markAllVertexDataChanged(start, end)`
A shorthand method to mark the position, texture co-ordinate and color buffers changed. The `start` and `end` parameters are optional and work the same way as with `markDataChanged()`.

`markIndexDataChanged(start, end)`
A shorthand method that is equivalent to `markDataChanged("indices", start, end)`.

`fillColor(r, g, b, a)`
A helper method to fill the entire color buffer with the same color repeated for every vertex. This can be useful if there is no color data provided for the mesh - filling the color buffer with opaque white (all parameters 1) will keep the original texture color. Note this method does not mark the buffer changed.

`release()`
Release this mesh data, freeing both the CPU and GPU memory associated with it. The mesh data cannot be accessed or drawn after releasing.